Empirical Properties of Dependency Trees in Natural Languages

Pujit Sharath Shetty

1 Motivation for the Research Problem

Natural language sentences can be represented as dependency trees, where words are nodes and syntactic relations define directed edges. This representation provides a simple and intuitive way to study sentence structure using graph-based properties such as depth, branching, and connectivity.

A key question is whether these structures are largely random, or whether they reflect deeper constraints imposed by language. In other words, if we fix sentence length, do we obtain something close to a random tree, or does natural language follow a more organized pattern?

This question is important for understanding how language is processed. If dependency trees consistently differ from random structures, it suggests that language is shaped by pressures such as minimizing cognitive effort and maintaining efficient communication.

To examine this, we compare real dependency trees with uniformly random trees of the same size. This provides a clear baseline: consistent deviations from randomness indicate the presence of underlying structural principles governing language.

Objective: To determine whether dependency trees in natural language exhibit systematic, non-random structural properties across multiple languages.

2 Hypotheses and Predictions

We test whether dependency trees in natural language differ systematically from random tree structures along three key dimensions: efficiency, structural organization, and global structure.

H1 (Efficiency): Natural language trees are more efficient in organizing dependencies.
Prediction: Dependency length (DL), intervener complexity (IC), and weighted intervener complexity (WIC) are lower in natural trees than in random trees.

H2 (Structural organization): Natural language trees favor shallower hierarchies with stronger local branching.
Prediction: Tree depth and average depth are lower, while maximum arity is higher in natural trees.

H3 (Global structure): Natural language trees differ from random trees in their global organization.
Prediction: Metrics such as path integration cost (PIC) and average LCA depth show systematic differences between natural and random trees.

3 Methods

We use dependency treebanks from the Surface-Syntactic Universal Dependencies (SUD) corpus. In this representation, each sentence forms a rooted directed tree, where words are nodes and syntactic dependencies define parent–child relations. We analyze multiple languages to examine whether the observed patterns are consistent across different linguistic systems.

For each sentence, we extract its dependency structure and compute a set of structural metrics corresponding to our hypotheses. These include depth-based measures (tree depth and average depth), branching properties (maximum arity and degree entropy), and efficiency (dependency length, defined as the linear distance between heads and dependents).

To capture structural interactions, we compute intervener complexity (IC) and weighted intervener complexity (WIC), which measure how many structurally relevant nodes intervene between related words. We also analyze global organization using path integration cost (PIC) and average LCA depth.

As a baseline, we generate uniformly random trees of the same size using Prüfer sequences, which produce labeled trees with equal probability. Each random tree is then rooted and directed to ensure comparability with dependency trees.

For each sentence of length n, we generate 20 random trees and average the metric values to reduce variability. All metrics are computed at the sentence level and then aggregated per language.

For metrics that depend on linear order (such as dependency length, IC, and WIC), real trees use the original word order of the sentence, while random trees are assigned positions using a uniformly random permutation. This provides a fully unconstrained baseline in which both structure and word order are randomized.

To assess statistical significance, we use paired t-tests (comparing each sentence with its corresponding random baseline) and Mann–Whitney U tests as a non-parametric alternative.

In addition, we perform a depth scaling analysis to examine how tree depth grows with sentence length, providing a complementary view of structural organization.

4 Experimental Setup

We analyze ten languages from the SUD corpus: English, French, German, Spanish, Italian, Russian, Chinese, Arabic, Hindi, and Japanese. Sentences with fewer than three tokens are excluded, as they do not yield meaningful tree structures.

The final dataset contains over 165,000 sentences. The number of sentences per language is shown below:

Language	Sentences
Arabic	5,979
German	13,771
Italian	12,871
English	11,400
Chinese	3,997
Japanese	7,045
French	14,448
Hindi	13,304
Spanish	14,182
Russian	69,047

Dependency trees are extracted directly from the CoNLL-U annotations using a consistent processing pipeline. For each sentence, we generate 20 random trees of the same size using Prüfer sequences and convert them into rooted directed trees.

All metrics are computed for both real and random trees at the sentence level and then aggregated per language, allowing for systematic cross-linguistic comparison.

5 Results

We compare dependency trees from natural language with uniformly random trees across ten languages. Across all metrics, natural trees consistently differ from random ones, indicating strong structural constraints. The clearest effects appear in efficiency-related measures, while global structural metrics show weaker but still systematic differences.

5.1 Dependency Length

Dependency length (DL) shows the strongest and most consistent effect. Natural trees have substantially lower DL than random trees across all languages.

The gap (random minus real) ranges from about 3.5 (Russian) to 9.7 (Arabic). At the sentence level, random trees have higher DL in over 97% of cases, often exceeding 99%. This reflects a strong and nearly universal tendency for related words to stay close in linear order.

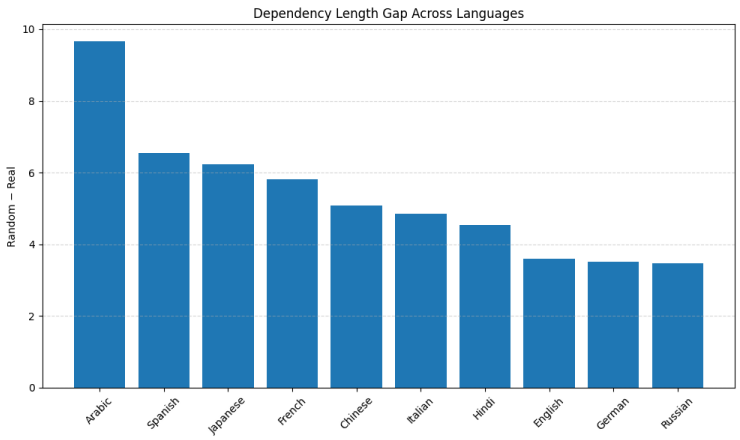

Figure 1: Dependency length difference (random minus real) across languages.

5.2 Structural Complexity (IC / WIC)

Intervener-based metrics show similarly strong patterns. Natural trees consistently have lower IC and WIC than random trees, with random trees exceeding them in over 98–100% of sentences.

This indicates that language avoids not only long dependencies, but also structurally complex configurations where multiple competing heads intervene. In other words, efficiency operates at both the linear and structural levels.

5.3 Tree Structure: Depth and Branching

Natural trees are generally shallower than random trees, with depth gaps ranging from about 0.24 to 2.98. Random trees are deeper in roughly 60–93% of sentences across languages.

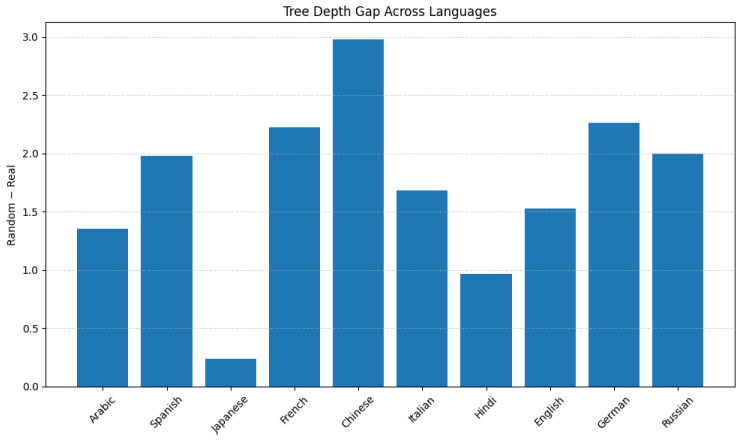

Figure 2: Depth difference (random minus real) across languages.

At the same time, natural trees exhibit higher branching (maximum arity), indicating that structure is more locally concentrated.

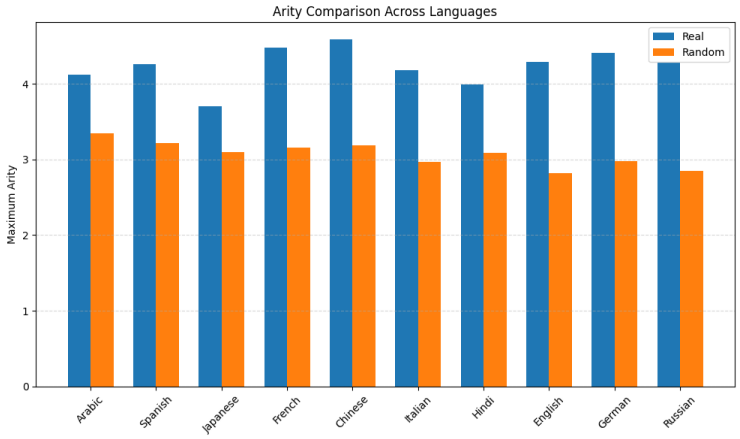

Figure 3: Maximum node arity in real vs random trees across languages.

Taken together, these results point to a clear trade-off: rather than forming deep hierarchical chains, natural language tends to organize structure through increased local branching around central nodes.

Degree entropy shows relatively small and inconsistent differences across languages. This suggests that while natural trees differ clearly in depth and branching magnitude, the variability of branching itself is not a primary distinguishing factor.

5.4 Global Structure: PIC and LCA

Average LCA depth is consistently lower in natural trees, indicating a more compact hierarchy where relationships between words are resolved closer to central nodes.

Path integration cost (PIC), however, shows a weaker and qualitatively different pattern. Natural trees often have slightly higher PIC, with gaps typically in the range of 0.3–1.0. This suggests that language does not strictly minimize global path-based cost, but instead balances it against stronger local constraints such as dependency length and structural interference.

5.5 Statistical Significance

All primary effects (DL, IC, WIC, depth, arity, and LCA) are highly significant across languages ($p \ll 0.001$) under both paired t-tests and Mann–Whitney U tests. Even weaker effects such as PIC and entropy remain statistically significant due to the large dataset.

5.6 Depth Scaling

We also examine how tree depth scales with sentence length. Random trees show stable scaling behavior, with exponents typically around 0.58–0.62.

In contrast, natural language trees show greater variation. Some languages exhibit slower growth, indicating tighter control over hierarchical expansion, while others are closer to the random baseline. This suggests that while structural constraints are broadly shared, the way hierarchical complexity develops with sentence length varies across languages.

5.7 Summary

Across all languages, the following patterns consistently emerge:

- Strong reduction in dependency length
- Strong reduction in intervener complexity
- Shallower trees with higher local branching
- More compact hierarchical organization (lower LCA depth)
- Weaker or mixed behavior in global structural cost (PIC)

Overall, these results show that natural language structure is systematically non-random and shaped by multiple interacting constraints, with efficiency and locality playing the dominant role.

6 Theoretical Implications

The results show that natural language dependency trees are not random, but exhibit clear and consistent structural patterns. These patterns appear across all languages analyzed, suggesting that they reflect general constraints on how language is organized rather than properties of specific datasets.

The most prominent effect is the reduction in dependency length. Across all languages, natural trees consistently keep related words closer together than random trees. This supports the view that language is shaped by processing efficiency, since shorter dependencies reduce memory load and support incremental interpretation.

Importantly, this pressure extends beyond linear distance. Intervener-based metrics (IC and WIC) show that natural language also avoids structurally dense regions of the tree. Dependencies tend to pass through areas with fewer competing nodes, indicating that language minimizes structural interference in addition to distance. In this sense, efficiency operates at both the linear and hierarchical levels.

At the same time, natural trees exhibit a characteristic global organization. They are generally shallower than random trees while also being more highly branched. Rather than forming deep

chains, structure is concentrated locally around central nodes. This reflects a clear trade-off: complexity is handled through local branching instead of deeper hierarchical nesting, allowing language to remain both compact and expressive.

However, not all aspects of structure are optimized in the same way. Path integration cost (PIC), which captures global structural load, does not consistently decrease and is often slightly higher in natural trees. This suggests that language is not optimizing a single global objective, but instead balancing multiple constraints, with local efficiency taking priority over global simplicity.

LCA-based analysis further supports this interpretation. Natural trees tend to have lower average LCA depth, meaning that relationships between words are resolved higher in the tree. This points to a more centralized and compact organization, where dependencies are integrated earlier rather than distributed across deeper levels.

The cross-linguistic consistency of these patterns is particularly notable. Despite differences in syntax and word order, all languages show strong reductions in dependency length and intervener complexity, along with similar depth–branching trade-offs. At the same time, the strength of these effects varies. This variation likely reflects typological differences: languages with more flexible word order or richer morphology can rely more on linear arrangement to maintain locality, while others may rely more heavily on structural organization, leading to differences in depth and branching patterns. This suggests that languages balance the same underlying constraints in different ways.

Overall, these findings suggest that natural language structure emerges from a balance between competing pressures. Dependency trees simultaneously minimize dependency distance, reduce structural interference, and maintain expressive flexibility. Rather than optimizing a single property, language appears to be shaped by interacting constraints, with efficiency and locality playing the dominant role.

6.1 Limitations

The random tree baseline used in this study is fully unconstrained and does not capture linguistic properties such as projectivity or word order preferences. As a result, some differences—particularly in dependency length and intervener complexity—may be amplified. Future work could explore more constrained baselines to better isolate the specific mechanisms shaping natural language structure.

7 Conclusion

This study shows that natural language dependency trees are systematically non-random. Across all languages analyzed, they exhibit shorter dependencies, lower structural complexity, and shallower yet more locally organized structures compared to random baselines.

Taken together, these patterns suggest that language is shaped by strong pressures toward efficiency and locality. Dependencies are kept short, structurally complex regions are avoided, and hierarchical organization is managed in a way that limits depth while maintaining expressive capacity.

At the same time, the results indicate that language does not optimize a single structural property. Instead, it reflects a balance between competing constraints, where local efficiency plays a dominant role while global structural properties are more weakly constrained. This leads to systems that are both efficient to process and flexible enough to support diverse linguistic structures.

Finally, the consistency of these patterns across typologically different languages suggests that these constraints are not language-specific, but reflect more general principles of human language organization.

Appendix

A Mathematical and Implementation Details

A.1 Random Tree Generation via Prüfer Sequences

Prüfer sequences provide a convenient way to represent labeled trees. There is a one-to-one correspondence between labeled trees on n nodes and sequences of length $n-2$ over the set $\{1, 2, \ldots, n\}$. As a result, the total number of labeled trees on n nodes is:

$$n^{n-2}$$

To generate a uniformly random labeled tree, we sample a sequence:

$$P = (p_1, p_2, \ldots, p_{n-2}), \quad p_i \sim \text{Uniform}\{1, \ldots, n\}$$

Because this mapping is bijective, sampling Prüfer sequences uniformly ensures that all labeled trees are equally likely. This makes Prüfer sequences a natural choice for constructing an unbiased random baseline.

A.1.1 Decoding Implementations

To convert a Prüfer sequence into a tree, we implemented multiple decoding strategies with time complexities of $O(n^2)$, $O(n \log n)$, and $O(n)$.

The $O(n^2)$ method repeatedly scans for the smallest available leaf, while the $O(n \log n)$ method uses a priority queue to track leaves efficiently. In the final pipeline, we use the $O(n)$ algorithm, which avoids explicit data structures by maintaining a degree array along with a pointer to track the smallest available leaf.

All implementations produce identical trees for a given sequence, but the linear-time version is significantly more efficient and is used throughout our experiments.

A.1.2 Decoding Algorithm (Linear Time)

Given a Prüfer sequence $P = (p_1, \ldots, p_{n-2})$, the corresponding tree can be constructed in $O(n)$ time using a degree array and a pointer-based leaf selection strategy.

The key idea is to maintain the degree of each node and repeatedly connect the smallest available leaf to the next element in the sequence.

- Initialize degrees:
$$\deg(v) = 1 + \text{count of } v \text{ in } P$$
- Find the smallest node with $\deg(v) = 1$ and set it as the current leaf
- For each p_i in P:
  - Add edge (leaf, p_i)
  - Decrease $\deg(p_i)$
  - Update the next leaf:
    - If p_i becomes a leaf and is smaller than the current pointer, select it
    - Otherwise, advance the pointer to find the next smallest leaf

- Finally, connect the last two remaining nodes

This approach avoids maintaining explicit data structures such as heaps and achieves linear time complexity.

Implementation. The following Python function implements the linear-time decoding algorithm:

```
def prufer_to_tree_n(prufer):
    n=len(prufer)+2
    degree=[1]*(n+1)
    for v in prufer:
        degree[v]+=1
    ptr=1
    while degree[ptr]!=1:
        ptr+=1
    leaf=ptr
    edges=[]
    for v in prufer:
        edges.append((leaf,v))
        degree[v]-=1
        if degree[v]==1 and v<ptr:
            leaf=v
        else:
            ptr+=1
            while ptr<=n and degree[ptr]!=1:
                ptr+=1
            leaf=ptr
    edges.append((leaf,n))
    return edges
```

A.2 Tree Orientation

The trees generated from Prüfer sequences are undirected. To make them comparable to dependency trees, they must be converted into rooted directed structures.

We do this by first selecting a root node r uniformly at random. A depth-first search (DFS) is then performed starting from r, and each edge (u, v) is oriented from parent to child based on the traversal.

This procedure produces a rooted directed tree that is structurally comparable to dependency trees.

A.3 Structural Metrics

Let $T = (V, E)$ be a rooted tree with $|V| = n$ and root r.

A.3.1 Tree Depth

Tree depth measures the maximum distance from the root to any node:

$$\text{depth}(T) = \max_{v \in V} d(r, v)$$

This captures the extent of hierarchical nesting in the tree.

Time complexity: $O(n)$ via a single DFS traversal.

A.3.2 Average Depth

Average depth measures the mean distance of nodes from the root:

$$\text{avg_depth}(T) = \frac{1}{n} \sum_{v \in V} d(r, v)$$

This provides a global view of how deep the tree is overall.
Time complexity: $O(n)$

A.3.3 Maximum Node Arity

Maximum arity is the largest number of children of any node:

$$\text{arity}(T) = \max_{v \in V} \deg^{+}(v)$$

This reflects how much branching occurs at a single point in the tree.
Time complexity: $O(n)$

A.3.4 Dependency Length

Dependency length (DL) measures how far apart syntactically related words are in linear order. For a dependency edge (i, j):

$$\ell_{ij} = |i - j|$$

The average dependency length of a tree is:

$$\text{DL}(T) = \frac{1}{n-1} \sum_{(i,j) \in E} |i - j|$$

Lower values indicate that dependencies are more local, which is typically associated with reduced processing cost.
Time complexity: $O(n)$

Implementation. For real dependency trees, word order is given by token positions in the sentence. For random trees, node positions are assigned using a uniformly random permutation.

```
def compute_real_dl(sentence):
    total=0
    count=0
    for token in sentence:
        if not isinstance(token["id"],int):
            continue
        h=token["head"]
        if h==0:
            continue
        total+=abs(token["id"]-h)
        count+=1
    return total/count if count>0 else 0
```

```python
def compute_random_tree_dl(children):
    nodes=list(children.keys())
    positions=list(range(1,len(nodes)+1))
    random.shuffle(positions)
    pos={node:positions[i] for i,node in enumerate(nodes)}
    total=0
    count=0
    for parent in children:
        for child in children[parent]:
            total+=abs(pos[parent]-pos[child])
            count+=1
    return total/count if count>0 else 0
```

A.3.5 Degree Distribution and Entropy

This measures how branching is distributed across nodes. Let $k_v = \deg^+(v)$ and:

$$p_k = \frac{|\{v \in V : k_v = k\}|}{n}$$

$$H = -\sum_k p_k \log p_k$$

Higher entropy indicates greater variability in branching.
Time complexity: $O(n)$

A.3.6 Intervener Complexity (IC)

Intervener Complexity (IC) measures how many structurally relevant nodes (i.e., nodes with outgoing edges) lie between a dependent and its head in linear order. For a dependency (h, d):

$$\mathrm{IC}(h, d) = |\{i : \min(h, d) < i < \max(h, d), \deg^+(i) > 0\}|$$

$$\mathrm{IC}(T) = \frac{1}{n-1} \sum \mathrm{IC}(h, d)$$

Intuitively, IC captures the amount of structural interference between related words. Higher values indicate that dependencies pass through regions with more competing heads.

Naive computation. A direct approach is to examine all nodes between h and d for every dependency and count how many of them are heads. This requires scanning the entire interval for each edge, resulting in $O(n^2)$ time in the worst case.

Optimized computation. We optimize this by precomputing a prefix sum over sentence positions, where each position stores whether the corresponding node is a head. This allows us to compute the number of intervening heads in constant time per dependency.
Time complexity: $O(n)$

Implementation (real trees).

```
def compute_ic(sentence):
    _,children=build_tree(sentence)
    head_flag={node:1 if len(children[node])>0 else 0 for node in children
        }
    prefix={}
    running=0
    for i in sorted(head_flag.keys()):
        running+=head_flag[i]
        prefix[i]=running
    total_ic=0
    count=0
    for token in sentence:
        if not isinstance(token["id"],int):
            continue
        h=token["head"]
        d=token["id"]
        if h==0:
            continue
        l,r=min(h,d),max(h,d)
        ic=0 if r-l<=1 else prefix.get(r-1,0)-prefix.get(l,0)
        total_ic+=ic
        count+=1
    return total_ic/count if count>0 else 0
```

Implementation (random trees).

```
def compute_random_tree_ic(children):
    nodes=list(children.keys())
    positions=list(range(1,len(nodes)+1))
    random.shuffle(positions)
    pos={node:positions[i] for i,node in enumerate(nodes)}
    is_head={node:len(children[node])>0 for node in children}
    total_ic=0
    count=0
    for parent in children:
        for child in children[parent]:
            h,d=pos[parent],pos[child]
            l,r=min(h,d),max(h,d)
            ic=0
            for node in nodes:
                p=pos[node]
                if l<p<r and is_head[node]:
                    ic+=1
            total_ic+=ic
            count+=1
    return total_ic/count if count>0 else 0
```

A.3.7 Weighted Intervener Complexity (WIC)

Weighted Intervener Complexity (WIC) extends IC by accounting for the importance of intervening nodes. Instead of simply counting intervening heads, each node is weighted by its outdegree. For

a dependency (h, d):

$$\mathrm{WIC}(h,d) = \sum_{i=\min(h,d)+1}^{\max(h,d)-1} \deg^+(i)$$

$$\mathrm{WIC}(T) = \frac{1}{n-1} \sum \mathrm{WIC}(h,d)$$

Intuitively, WIC captures not only how many nodes intervene, but how structurally significant they are. Intervening nodes with many dependents contribute more to the overall complexity.

Naive computation. A direct approach scans all nodes between h and d for each dependency and sums their outdegrees. This results in $O(n^2)$ time in the worst case.

Optimized computation. We optimize this using prefix sums over node weights (outdegrees). By precomputing cumulative sums, the total contribution of intervening nodes can be obtained in constant time per dependency.

Time complexity: $O(n)$

Implementation (real trees).

```
def compute_wic(sentence):
    _,children=build_tree(sentence)
    weights={node:len(children[node]) for node in children}
    prefix={}
    running=0
    for i in sorted(weights.keys()):
        running+=weights[i]
        prefix[i]=running
    total_wic=0
    count=0
    for token in sentence:
        if not isinstance(token["id"],int):
            continue
        h=token["head"]
        d=token["id"]
        if h==0:
            continue
        l,r=min(h,d),max(h,d)
        wic=0 if r-l<=1 else prefix.get(r-1,0)-prefix.get(l,0)
        total_wic+=wic
        count+=1
    return total_wic/count if count>0 else 0
```

Implementation (random trees).

```
def compute_random_tree_wic(children):
    nodes=list(children.keys())
    positions=list(range(1,len(nodes)+1))
    random.shuffle(positions)
    pos={node:positions[i] for i,node in enumerate(nodes)}
    outdegree={node:len(children[node]) for node in children}
    total_wic=0
    count=0
```

```
    for parent in children:
        for child in children[parent]:
            h,d=pos[parent],pos[child]
            l,r=min(h,d),max(h,d)
            wic=0
            for node in nodes:
                p=pos[node]
                if l<p<r and outdegree[node]>0:
                    wic+=outdegree[node]
            total_wic+=wic
            count+=1
    return total_wic/count if count>0 else 0
```

A.3.8 Path Integration Cost (PIC)

Path Integration Cost (PIC) measures the total structural load encountered along paths between all pairs of nodes. For a tree T:

$$\mathrm{PIC}(T) = \frac{1}{\binom{n}{2}} \sum_{i<j} \sum_{v \in \mathrm{path}(i,j)} \deg^+(v)$$

Intuitively, PIC captures how much structural "weight" is encountered when traversing the tree. Nodes with many children act as hubs and contribute more to paths passing through them.

Naive computation. A direct approach is to consider all pairs of nodes (i, j), explicitly compute the path between them, and sum the contributions of all nodes along the path. Since there are $O(n^2)$ pairs and each path can take $O(n)$ time, this results in $O(n^3)$ time in the worst case.

Optimized computation. We instead compute PIC using node contributions. Removing a node v partitions the tree into connected components of sizes $s_1, s_2, \ldots, s_k$. Any pair of nodes that lies in different components must have its path pass through v.

The number of such pairs is:

$$\frac{(\sum s_i)^2 - \sum s_i^2}{2}$$

Each such pair contributes $\deg^+(v)$ to the total. Summing over all nodes allows us to compute PIC in linear time.

Time complexity: $O(n)$

Implementation.

```
def compute_pic_all_pairs_from_children(children):
    nodes=list(children.keys())
    n=len(nodes)
    all_children=set(c for v in children for c in children[v])
    root=next(v for v in children if v not in all_children)
    f={node:len(children[node]) for node in children}
    subtree={}
    def dfs(node):
        size=1
        for child in children[node]:
            size+=dfs(child)
```

```
        subtree[node]=size
        return size
    dfs(root)
    total_pic=0
    for v in children:
        components=[subtree[c] for c in children[v]]
        parent_side=n-subtree[v]
        if parent_side>0:
            components.append(parent_side)
        total=n-1
        sum_sq=sum(x*x for x in components)
        pairs=(total*total - sum_sq)//2
        total_pic+=f[v]*pairs
    total_pairs=n*(n-1)//2
    return total_pic/total_pairs if total_pairs>0 else 0
```

A.3.9 Average LCA Depth

Average LCA depth measures how deep in the tree pairs of nodes are connected. For a tree T:

$$\mathrm{LCA}(T) = \frac{1}{\binom{n}{2}} \sum_{i<j} d(r, \mathrm{LCA}(i,j))$$

Higher values indicate that nodes tend to connect through deeper parts of the hierarchy, reflecting stronger hierarchical nesting.

Naive computation. A direct approach considers all pairs of nodes (i, j), computes their lowest common ancestor, and sums the depth of that node. Since there are $O(n^2)$ pairs and each LCA query can take $O(n)$ time in a tree, this leads to $O(n^3)$ time in the worst case.

Optimized computation. We instead compute LCA depth using node contributions. For a node v, removing it partitions the tree into components of sizes $s_1, \ldots, s_k$. Any pair of nodes that lies in different components has v as their lowest common ancestor.

The number of such pairs is:

$$\frac{(\sum s_i)^2 - \sum s_i^2}{2}$$

In addition, for every node u in the subtree of v, the pair (v, u) also has LCA equal to v, contributing:

$$\mathrm{subtree}(v) - 1$$

Combining both contributions:

$$\mathrm{pairs}(v) = \frac{(\sum s_i)^2 - \sum s_i^2}{2} + (\mathrm{subtree}(v) - 1)$$

Each such pair contributes $d(r, v)$ to the total. Summing over all nodes:

$$\mathrm{LCA}(T) = \frac{1}{\binom{n}{2}} \sum_{v \in V} d(r, v) \cdot \mathrm{pairs}(v)$$

Time complexity: $O(n)$

Implementation.

```
def compute_avg_lca_depth(children):
    nodes=list(children.keys())
    n=len(nodes)
    all_children=set(c for v in children for c in children[v])
    root=next(v for v in children if v not in all_children)
    subtree={}
    depth={}
    def dfs(node,d):
        depth[node]=d
        size=1
        for child in children[node]:
            size+=dfs(child,d+1)
        subtree[node]=size
        return size
    dfs(root,0)
    total=0
    for v in children:
        components=[subtree[c] for c in children[v]]
        total_size=sum(components)
        sum_sq=sum(x*x for x in components)
        pairs=(total_size*total_size - sum_sq)//2
        pairs+=(subtree[v]-1)
        total+=depth[v]*pairs
    total_pairs=n*(n-1)//2
    return total/total_pairs if total_pairs>0 else 0
```

A.4 Depth Scaling

To analyze how hierarchical structure grows with sentence length, we model tree depth as a power-law function of sentence length:

$$\text{depth} \sim n^b$$

Taking logarithms:

$$\log(\text{depth}) = b \log n + c$$

This allows us to estimate the scaling exponent b using linear regression in log-space:

$$b = \frac{\text{Cov}(\log n, \log \text{depth})}{\text{Var}(\log n)}$$

The exponent b captures how quickly depth increases with sentence length, providing a global view of structural growth.

A.5 Statistical Testing

Let X_i and Y_i denote the values of a metric for real and random trees for the same sentence.

A.5.1 Paired t-test

We compute differences:

$$d_i = X_i - Y_i$$

The test statistic is:

$$t = \frac{\bar{d}}{s_d/\sqrt{n}}$$

where:

$$\bar{d} = \frac{1}{n}\sum d_i, \quad s_d^2 = \frac{1}{n-1}\sum (d_i - \bar{d})^2$$

This test evaluates whether the mean difference between paired observations is significantly different from zero.

A.5.2 Mann–Whitney U Test

The Mann–Whitney U test compares the rank distributions of two samples without assuming normality.

Let $R(X)$ be the sum of ranks of sample X when all observations from both groups are combined and ranked. The test statistic is:

$$U = R(X) - \frac{n_X(n_X+1)}{2}$$

An equivalent formulation is:

$$U = \sum_{i=1}^{n_X}\sum_{j=1}^{n_Y} \mathbf{1}(X_i > Y_j)$$

where $\mathbf{1}(\cdot)$ is the indicator function.

For large samples, U is approximately normally distributed, enabling significance testing.

A.6 Reproducibility

All experiments are implemented in Python with a fixed random seed to ensure reproducibility. The complete codebase, including data processing and metric computation, is available at:

`https://github.com/Quasar4606/Dependency-tree-structure`

B Additional Results and Visualizations

This section presents additional tables and figures that complement the main analysis. These results provide further evidence for the patterns observed across languages.

B.1 Cross-Language Comparisons

All languages show strong reductions in dependency length and intervener complexity, while global metrics such as PIC exhibit weaker and more variable behavior.

Table 1 summarizes key differences between real and random trees across languages.

Language	Sent	DL	IC	WIC	Depth	Arity	PIC	LCA
Arabic	5979	9.66	6.16	9.58	1.36	-0.78	0.32	0.85
Spanish	14182	6.56	4.32	6.56	1.98	-1.04	0.87	1.18
Japanese	7045	6.23	3.96	6.08	0.24	-0.60	0.28	0.34
French	14448	5.82	3.88	5.85	2.22	-1.33	0.99	1.21
Chinese	3997	5.09	3.48	5.28	2.98	-1.40	0.73	1.38
Italian	12871	4.85	3.26	4.90	1.68	-1.21	0.86	0.99
Hindi	13304	4.55	2.89	4.46	0.96	-0.91	0.40	0.71
English	11400	3.61	2.54	3.77	1.53	-1.46	0.92	0.84
German	13771	3.51	2.56	3.80	2.26	-1.44	0.97	1.10
Russian	69047	3.48	2.46	3.65	2.00	-1.44	0.92	0.96

Table 1: Cross-language comparison of key metrics (gaps between random and real trees). Positive values indicate higher values in random trees, except for PIC where the gap is defined as (real minus random).

B.2 Additional Figures

Efficiency Metrics. Figures 4, 5, and 6 show strong and consistent reductions in dependency length and intervener-based complexity across all languages.

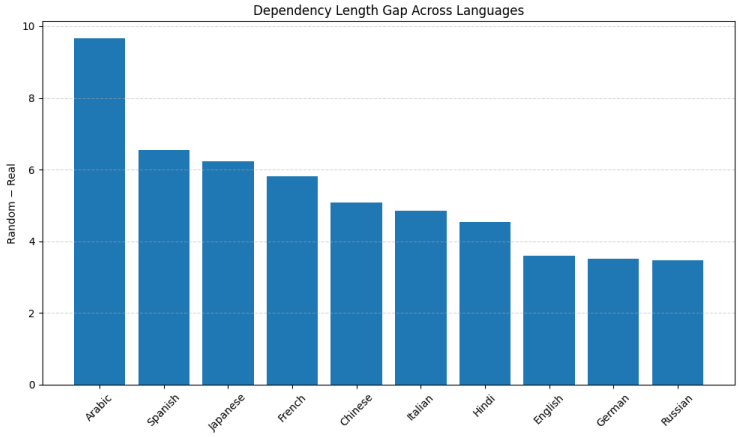

Figure 4: Dependency length gap (random minus real) across languages.

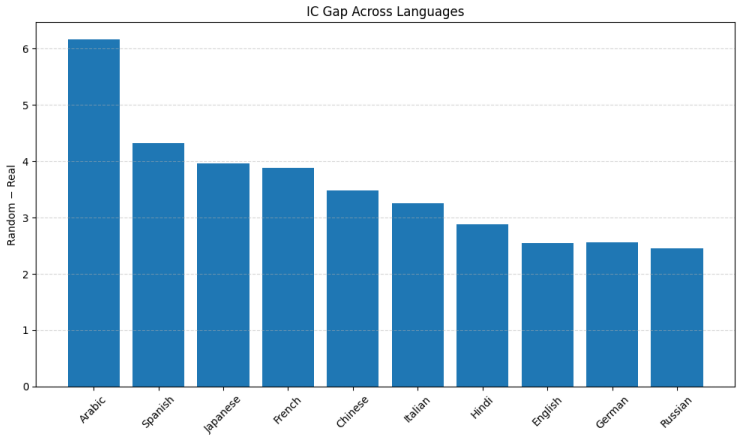

Figure 5: Intervener complexity gap across languages.

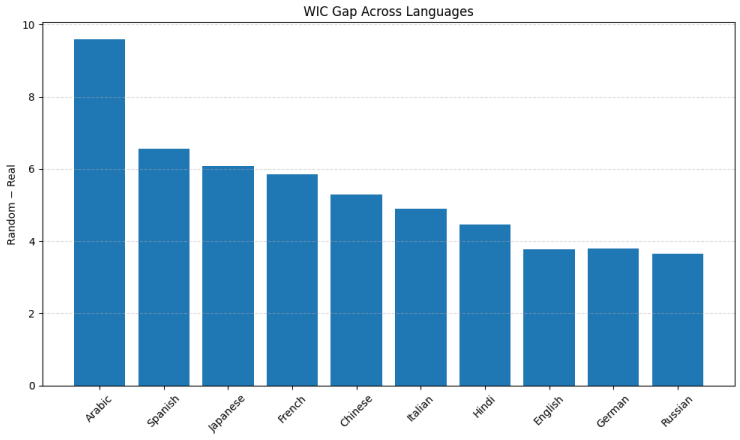

Figure 6: Weighted intervener complexity gap across languages.

Hierarchy and Branching. Figures 7 and 8 highlight the depth–branching trade-off: natural trees are shallower while exhibiting higher local branching.

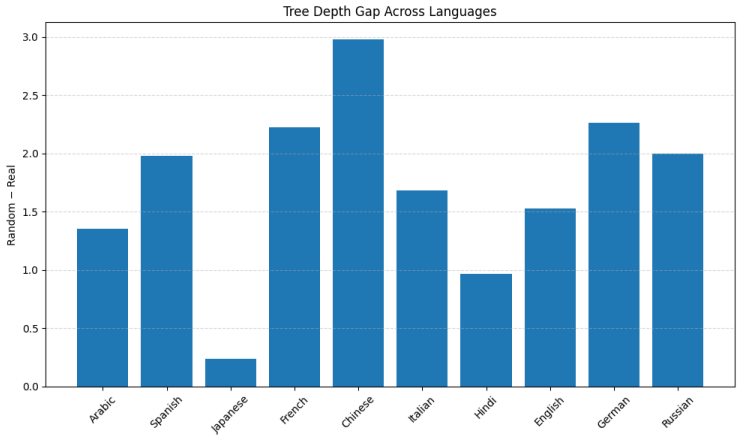

Figure 7: Tree depth gap (random minus real) across languages.

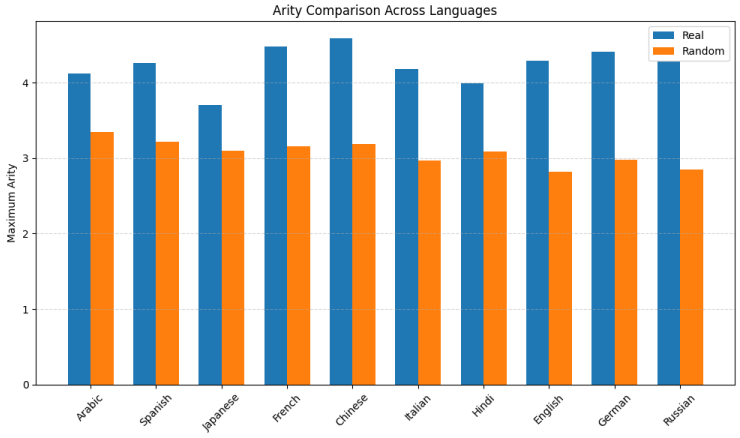

Figure 8: Maximum node arity in real vs random trees across languages.

Global Structure. Figures 9 and 10 show that global structural properties behave differently: LCA depth is consistently lower in natural trees, while PIC exhibits weaker and more variable differences.

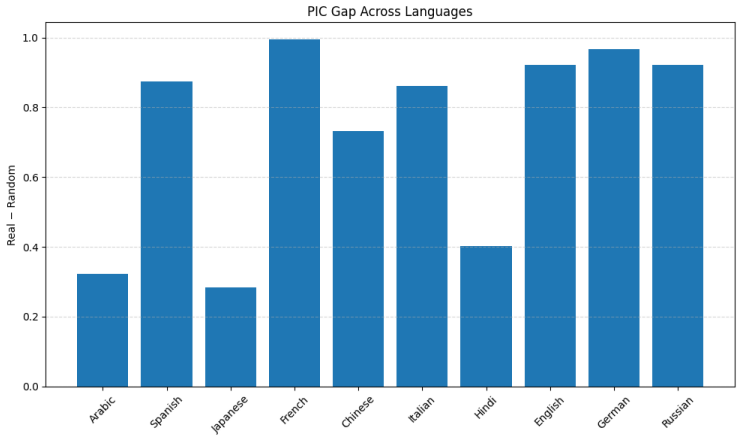

Figure 9: Path integration cost difference (real minus random) across languages.

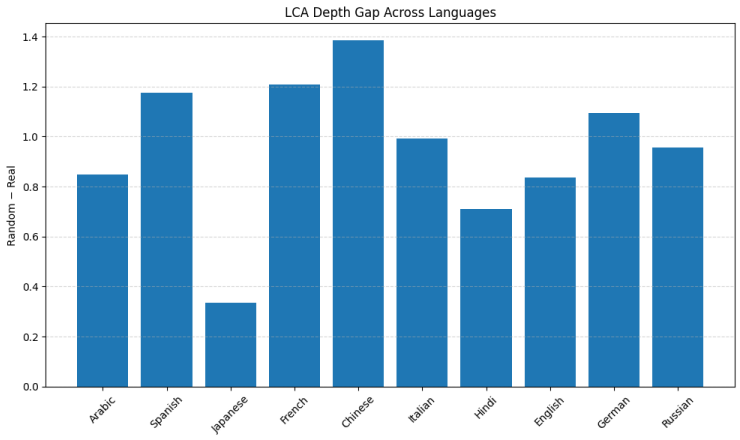

Figure 10: Average LCA depth gap (random minus real) across languages.

B.3 Depth Scaling Analysis

Real trees show more variability in scaling behavior, while random trees exhibit more stable exponents across languages.

Figure 11 shows how tree depth scales with sentence length in log-log space.

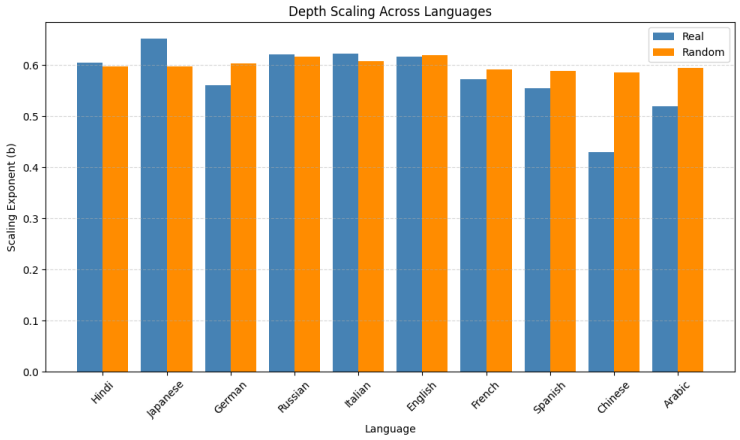

Figure 11: Depth scaling across languages (log-log plot).

Table 2 reports the estimated scaling exponents for real and random trees.

Language	Real b	Random b
Hindi	0.605	0.598
Japanese	0.651	0.598
German	0.561	0.603
Russian	0.621	0.617
Italian	0.623	0.607
English	0.616	0.619
French	0.573	0.591
Spanish	0.554	0.589
Chinese	0.430	0.586
Arabic	0.519	0.594

Table 2: Scaling exponents for depth as a function of sentence length.

B.4 Statistical Significance

All primary metrics show highly significant differences between real and random trees across languages.

Language	DL	IC	WIC	Depth	Arity	PIC	LCA
Hindi	4.54 (0)	2.89 (0)	4.46 (0)	0.96 (0)	-0.91 (0)	0.40 (0)	0.71 (0)
Japanese	6.23 (0)	3.96 (0)	6.08 (0)	0.24 (1e-17)	-0.60 (0)	0.28 (1e-147)	0.34 (0)
German	3.51 (0)	2.56 (0)	3.80 (0)	2.26 (0)	-1.44 (0)	0.97 (0)	1.09 (0)
Russian	3.48 (0)	2.46 (0)	3.65 (0)	2.00 (0)	-1.44 (0)	0.92 (0)	0.96 (0)
Italian	4.85 (0)	3.26 (0)	4.90 (0)	1.68 (0)	-1.21 (0)	0.86 (0)	0.99 (0)
English	3.61 (0)	2.54 (0)	3.77 (0)	1.53 (0)	-1.46 (0)	0.92 (0)	0.83 (0)
French	5.82 (0)	3.88 (0)	5.85 (0)	2.22 (0)	-1.33 (0)	0.99 (0)	1.21 (0)
Spanish	6.56 (0)	4.32 (0)	6.56 (0)	1.98 (0)	-1.04 (0)	0.87 (0)	1.18 (0)
Chinese	5.09 (0)	3.48 (0)	5.28 (0)	2.98 (0)	-1.40 (0)	0.73 (0)	1.38 (0)
Arabic	9.66 (0)	6.16 (0)	9.58 (0)	1.36 (1e-202)	-0.78 (0)	0.32 (1e-110)	0.85 (0)

Table 3: Differences (with p-values in parentheses) between real and random trees across languages.